

ERP Module Access Outage - Incident Report

Project: Indus ERP (INDUS OS)

Primary date: 18 September 2026

Status: Resolved (platform healthy; app defenses in place; remaining items are deploy/commit of local fixes where not yet live)

Audience: Product, IT, engineering

This report covers three instances of the same event timeline:

1. Before the incident - normal design and assumptions
2. During the incident - what broke, where it showed up, and how it was handled
3. After the fixes - what changed and how we prevent a repeat

Each instance is answered with Why / When / Where / What / How.

Executive summary

Users (including Super Admins) temporarily lost ERP module navigation and saw settings-only / "no access" behaviour. Database rows for assigned access were largely still correct. The outage was driven by Supabase PostgREST (data API) becoming unhealthy, while Auth continued to work. The frontend then fell back to a "pending signup" profile stub and, in some paths, kept that stub sticky, which stripped the UI of modules even after the database recovered-until caches were cleared and/or defenses were applied.

Separately, while building clearer platform health visibility, we found two undeployed Edge Functions used for bulk user management; those were deployed the same day.

Instance 1 - Before the incident

Why

The product needed a safe path for:

- New logins that exist in Auth but are not yet assigned a team / modules
- Preventing accidental privilege escalation during self-signup or incomplete profile creation
- Keeping Super Admin and assigned users on their real module set once IT grants access

So the system introduced:

Concept	Intent
---------	--------

profiles.module_access_pending	true = "login exists, wait for IT to assign access"
Settings-only / pending stub	Temporary UI when a real profile cannot be loaded yet
login-check Edge Function	Ensure a profile row exists on login
Client profile cache (Auth context)	Faster loads; survive brief network blips

Assumption (pre-incident): If Auth works and a profile row exists with team/modules/role, the SPA will always show those modules. Pending was treated as a strong "settings only" signal, without always checking whether the user already had assigned access.

When

Ongoing design state up through 17 September 2026 (including User Management / P&L access work in commit 7b1c55e) and into the morning of 18 September 2026, before PostgREST degradation became user-visible.

Where

Layer	Location
Database	public.profiles - role, team, allowed_modules, allowed_sub_modules, module_access_pending
Edge	supabase/functions/login-check (and related admin user functions)
SPA access math	src/config/roles.js -> getAccessibleModules
Session / profile sync	src/contexts/AuthContext.jsx, src/lib/loginFlow.js, src/lib/authSessionUtils.js, src/lib/safeSelfProfile.js
Navigation	src/contexts/Layout.jsx
Hosting / data plane	Supabase project wbyzhknaqcjqqtwpupl - Auth vs REST (PostgREST)

What

Normal happy path

1. User signs in -> Auth issues session.
2. App loads profile from REST / login-check.
3. getAccessibleModules(profile) builds the module set from role, team, and allowed modules.
4. Layout shows those modules.

Pending path (by design)

1. New or unassigned user -> module_access_pending = true, empty modules.
2. UI shows settings-only / limited access until an admin assigns team/modules and clears pending.

Resilience path (by design, but incomplete)

1. If profile fetch fails, app may build a safe signup stub (safeSelfSignupProfileFields) with module_access_pending: true.
2. That stub was meant for "no profile yet," not for "DB is down for an already-assigned user."

How

Access was computed client-side from the in-memory / cached profile. There was no independent System status view that clearly separated:

- Auth healthy vs Database (REST) healthy
- "Slow" vs "Down" per service group

Operators relied on console errors, Supabase dashboard, and user reports.

Instance 2 - During the incident

Why

Two failure modes stacked:

A. Platform (root trigger)

Supabase PostgREST / database API became slow or unavailable (timeouts; stuck idle-in-transaction style symptoms; later project recovery / restart). Auth could still succeed, so users could "sign in" while profile reads/writes timed out or returned 504.

B. Application (amplifier)

When REST failed:

1. Profile sync could not refresh the real row.
2. Code paths could apply or retain an empty pending stub (settings-only).
3. Access logic treated module_access_pending === true as hide modules, even when the real DB row still had Super Admin / team / allowed_modules.
4. Cached / sticky session state made the stub feel permanent until hard refresh, storage clear, or successful re-sync.

So users experienced "everyone lost modules" even though assignments in profiles were often still present.

Secondary symptoms the same day:

- Invalid refresh token / session churn when Auth recovery and client storage got out of sync

- Confusing console noise while probing APIs
- Health checks showing Edge Functions admin-bulk-create-users / admin-bulk-delete-users as Down (they were never deployed, not corrupted by the outage)

When

18 September 2026 (local / IST working day), roughly:

Phase	What happened
Onset	Users report all ERP modules gone; Settings-only / no access
Investigation	DB check: e.g. Super Admin (super_admin_pro) still correct; module_access_pending false on that row - proves UI != DB
Parallel failures	Auth OK; REST timeouts / 504; refresh token errors
Platform ops	Paid plan: Pause unavailable; recovery via project restart / stuck query termination as applicable
Stabilisation	REST healthy again -> modules return once clients re-fetch real profiles
Same-day follow-ups	App defenses coded; System status UI; undeployed bulk Edge Functions deployed

Exact wall-clock start/end for PostgREST may be confirmed in Supabase project logs / metrics for that day.

Where

Symptom	Where it appeared
No modules in sidebar	Browser SPA (Layout + getAccessibleModules)
Login succeeds but empty ERP	Auth OK; REST path failing
Sticky "no access" after DB recovered	Client profile cache / AuthContext merge rules
504 / timeouts	Calls to Supabase REST (/rest/v1/...), config/profile fetches
Invalid refresh token	Browser storage + Auth refresh
Operator confusion	No plain-language "Database down vs Auth up" board until System status work
Bulk user Edge "Down"	Direct browser -> functions/v1/admin-bulk-* (404 / CORS on preflight because not deployed)

Not the primary failure location: Role tables wholesale wiped; accidental mass revoke of allowed_modules for all users (investigation contradicted that for key accounts).

What

Observable behaviour:

1. Signed-in users see little or no module navigation.
2. Super Admins affected the same way as executives - strong signal of client pending/stub logic, not "lost

Super Admin in DB."

3. Clearing session / waiting for REST recovery / hard refresh restored access when the platform was healthy again.
4. Local engineering changes were not yet required to be live for recovery once REST recovered; they harden against the next event.

How (response during the event)

1. Verify data, don't guess UI - service-role / SQL checks on profiles for affected users.
2. Split Auth vs REST - confirm Auth health endpoint vs REST timeouts.
3. Platform - inspect stuck sessions / idle transactions; restart project if needed (paid: use Restart, not Pause).
4. Client - clear bad session/local profile cache when refresh tokens are invalid.
5. Engineering - implement defenses so a future REST blip cannot overwrite a good profile with a pending stub; add System status for non-technical operators.
6. Deploy gap found via health UI - deploy admin-bulk-create-users and admin-bulk-delete-users.

Instance 3 - After the fixes

Why

Goals after recovery:

1. Never hide assigned access because of a stale or stub module_access_pending.
2. Never replace a real profile with an empty pending signup stub when sync fails.
3. Make login-check safe so upserts do not wipe existing profile fields (ignoreDuplicates / conflict behaviour).
4. Give operators a simple status board (Healthy / Needs attention / Problems) with click-through to which checks are slow or down.
5. Close real deploy gaps (bulk Edge Functions) exposed by that board.

When

Work	Timing
Access defenses + restore SQL migration	18 Sep 2026 (local changes; commit/deploy to production as release process allows)

System status (API health) UX	Same day - overview cards, detail popup (portal to document body)
Probe noise reduction	Same day - avoid POST {} validation 400 spam; treat reachability correctly
Edge Functions deploy	18 Sep 2026 ~06:10 UTC - admin-bulk-create-users, admin-bulk-delete-users ACTIVE
User confirmation	Later same morning IST - "error fixed and up running"

Where (artefacts)

Change	Path / target
Pending vs assigned access	src/config/roles.js - profileHasAssignedAccess, isEmptyPendingAccessStub, getAccessibleModules
Auth sync / no stub overwrite	src/context/AuthContext.jsx, src/lib/loginFlow.js, src/lib/authSessionUtils.js
Soft-fail config / session	src/lib/supabase.js and related session helpers
Login-check upsert safety	supabase/functions/login-check/index.ts (ignoreDuplicates: true on conflict)
Data cleanup (optional run)	supabase/migrations/20260918100000_restore_assigned_module_access.sql - clears pending only where team/modules/Super Admin already exist
Tests	tests/rolesAccess.test.js (and related)
System status UI	src/pages/apiMonitoring/* - overview, card click modal, probe handlers
Edge deploy + JWT config	Supabase project functions; supabase/config.toml verify_jwt = false for bulk admin functions
Layout / P&L visibility	src/context/Layout.jsx adjustments as part of access hardening

What

Application behaviour now

- If a profile has Super Admin / team / allowed modules -> show that access even if module_access_pending was incorrectly true.
- Empty pending stub is recognised and must not replace a richer cached/real profile.
- Login-check should not silently clobber existing profile rows on conflict.
- Operators open System status, click Business tools (etc.), and see named Down/Slow checks.
- Bulk create/delete Edge Functions are deployed and reachable; health no longer falsely flags them as missing.

Platform behaviour

- After REST recovered, assigned users see modules again without needing a mass re-grant of permissions.

How (operate going forward)

Immediate ops checklist (if modules vanish again)

1. Open System status - is Database red while Sign-in is green? -> treat as REST/platform, not "everyone's roles were deleted."
2. In Supabase: Auth health vs REST; look for stuck transactions / high latency; Restart project if needed (paid).
3. Spot-check profiles for a known Super Admin: role/team/modules vs UI.
4. Ask affected users to Refresh all / hard refresh; clear site storage only if refresh token errors persist.
5. If pending flags are stuck in DB for assigned users, run (or re-run) the restore migration logic - it only clears pending where access is already assigned.

Release checklist

- [] Commit and deploy SPA access defenses to the environment users hit.
- [] Deploy login-check if the ignoreDuplicates change is not yet live.
- [] Apply 20260918100000_restore_assigned_module_access.sql on environments that still have stuck pending flags.
- [] Keep Edge Function deploys in the release checklist when new admin functions are added (do not wait for a health red).

Prevention principles

Principle	Practice
Auth != Database	Monitor both; UI should say "data service slow/down," not only "login failed."
Pending is for unassigned only	Never let pending override Super Admin or already-assigned modules.
Fail closed on privileges, fail open on display of known grants	Don't invent modules; don't erase known grants on timeout.
Stub is not truth	Never persist a signup stub over a previously good profile.
Deploy what you monitor	Health checks that point at Edge Functions imply those functions must be deployed.

Root cause statement (one paragraph)

Root cause: Supabase data API (PostgREST) degradation while Auth remained available.

Contributing cause: Client access and profile-sync logic treated failed profile loads / pending stubs as "settings-only," which hid modules for users who still had valid assignments in profiles.

Resolution: Platform REST recovery restored data plane; application defenses ensure pending/stubs cannot strip assigned access; System status improves detection; missing bulk Edge Functions were deployed.

Appendix A - SQL restore (intent)

Migration 20260918100000_restore_assigned_module_access.sql:

- Sets module_access_pending = false
- Only where the row already has Super Admin role, non-empty team, or non-empty allowed_modules / allowed_sub_modules
- Does not grant new modules or change roles

Appendix B - Edge Functions deployed during follow-up

Function	Status before	Action
admin-bulk-create-users	Not listed / unreachable (CORS on preflight)	Deployed --no-verify-jwt
admin-bulk-delete-users	Same	Deployed --no-verify-jwt

Auth is still validated inside the function; gateway JWT verify remains off for ES256 / preflight reliability (same pattern as other admin functions).

Appendix C - Related commits / references

- Pre-incident access work: 7b1c55e (2026-09-17) - User Management / P&L access
- Same-day follow-up commit on branch history: 2b85ad9 (2026-09-18) - security / API health related
- Local uncommitted / in-progress hardening from this incident may still need explicit production deploy - verify before declaring all environments fully patched.

Document prepared from investigation and remediation on 18 September 2026.